

Mini-RISC CPU Simulator Code

main.cpp

```
#include <bits/stdc++.h>
using namespace std;

#include "alu.hpp"

class cpu
{
public:
    uint8_t r[4]{0}, acc = 0, pc = 0, ir = 0;
    bool z = 0, c = 0;
};

enum State
{
    FETCH,
    DECODE,
    EXEC,
    WB,
    HALT
};

struct Control
{
    bool PC_write = 0, IR_write = 0, ACC_write = 0, REG_write = 0;
    bool MEM_read = 0, PC_src = 0;
    uint8_t ALU_op = 0;
};

uint8_t mem[256];
ofstream log_file;

/* ----- Utilities ----- */

string state_name(State s)
{
    if (s == FETCH)
        return "FETCH";
    if (s == DECODE)
        return "DECODE";
    if (s == EXEC)
        return "EXEC";
    if (s == WB)
        return "WB";
    return "HALT";
}
```

```

void open_log()
{
    log_file.open("cpu_trace.txt", ios::app);
}

void reset_cpu(cpu &c)
{
    if (log_file.is_open())
        log_file.close();
    remove("cpu_trace.txt");
    open_log();

    memset(c.r, 0, sizeof(c.r));
    c.acc = c.pc = c.ir = 0;
    c.z = c.c = 0;
    memset(mem, 0, sizeof(mem));
}

/* ----- Instruction Decode ----- */

string decode(uint8_t ir)
{
    int op = (ir >> 4) & 0xF, r = ir & 0xF;
    switch (op)
    {
        case 0x0:
            return "NOP";
        case 0x1:
            return "LOAD R" + to_string(r);
        case 0x2:
            return "STORE R" + to_string(r);
        case 0x3:
            return "MOVI " + to_string(r);
        case 0x4:
            return "ADD R" + to_string(r);
        case 0x5:
            return "SUB R" + to_string(r);
        case 0x6:
            return "MUL R" + to_string(r);
        case 0x7:
            return "DIV R" + to_string(r);
        case 0x8:
            return "POW R" + to_string(r);
        case 0x9:
            return "CMP R" + to_string(r);
        case 0xA:
            return "JMP " + to_string(r);
        case 0xB:
            return "JZ " + to_string(r);
        case 0xC:
            return "JC " + to_string(r);
        case 0xF:
            return "HALT";
    }
}

```

```

}
return "???" ;
}

/* ----- Logging ----- */

void log_state(cpu &c, Control &ctrl, State s)
{
    time_t now = time(0);
    tm *t = localtime(&now);

    log_file << "[" << setw(2) << setfill('0') << t->tm_hour << ":" << setw(2) << t->tm_min
    << ":" << setw(2) << t->tm_sec << "]" << " ";
    log_file << state_name(s) << " | ";
    log_file << "PC=" << (int)c.pc << " IR=0x" << hex << setw(2) << (int)c.ir << dec << " ("
    << decode(c.ir) << ") | ";
    log_file << "R0=" << (int)c.r[0] << " R1=" << (int)c.r[1] << " R2=" << (int)c.r[2] << "
    R3=" << (int)c.r[3] << " | ";
    log_file << "ACC=" << (int)c.acc << " Z=" << c.z << " C=" << c.c << " | CTRL: ";

    if (ctrl.MEM_read)
        log_file << "MEMR ";
    if (ctrl.IR_write)
        log_file << "IRW ";
    if (ctrl.PC_write)
        log_file << "PCW ";
    if (ctrl.PC_src)
        log_file << "PCSRC ";
    if (ctrl.ACC_write)
        log_file << "ACCW ";
    if (ctrl.REG_write)
        log_file << "REGW ";
    if (ctrl.ALU_op)
        log_file << "ALU=" << hex << (int)ctrl.ALU_op << dec << " ";
    log_file << endl;
}

/* ----- Control Unit ----- */

void generate_control(cpu &c, State s, Control &ctrl)
{
    uint8_t opcode = (c.ir >> 4) & 0xF;

    if (s == FETCH)
    {
        ctrl.MEM_read = 1;
        ctrl.IR_write = 1;
        ctrl.PC_write = 1;
    }
    else if (s == EXEC)
    {
        if (opcode == 0x3 || opcode == 0x1)
            ctrl.ACC_write = 1;
        else if (opcode >= 0x4 && opcode <= 0x9)

```

```

        ctrl.ALU_op = opcode;
    else if (opcode == 0x2)
        ctrl.REG_write = 1;
    else if (opcode == 0xA)
        ctrl.PC_write = ctrl.PC_src = 1;
    else if (opcode == 0xB && c.z)
        ctrl.PC_write = ctrl.PC_src = 1;
    else if (opcode == 0xC && c.c)
        ctrl.PC_write = ctrl.PC_src = 1;
}
else if (s == WB)
{
    if (opcode ≥ 0x4 && opcode ≤ 0x8)
        ctrl.ACC_write = 1;
}
}

/* ----- Datapath ----- */

void apply_control(cpu &c, Control &ctrl)
{
    uint8_t op = c.ir & 0xF;
    static uint8_t alu_out;

    if (ctrl.MEM_read && ctrl.IR_write)
        c.ir = mem[c.pc];
    if (ctrl.PC_write)
        c.pc = ctrl.PC_src ? op : c.pc + 1;

    if (ctrl.ALU_op)
    {
        auto r = (ctrl.ALU_op == 0x4) ? alu_add(c.acc, c.r[op])
            : (ctrl.ALU_op == 0x5) ? alu_sub(c.acc, c.r[op])
            : (ctrl.ALU_op == 0x6) ? alu_mul(c.acc, c.r[op])
            : (ctrl.ALU_op == 0x7) ? alu_div(c.acc, c.r[op], *(bool *)0)
            : (ctrl.ALU_op == 0x8) ? alu_pow(c.acc, c.r[op])
            : alu_cmp(c.acc, c.r[op]);
        alu_out = r.value;
        c.z = r.Z;
        c.c = r.C;
    }

    if (ctrl.ACC_write)
    {
        if ((c.ir >> 4) == 0x3)
            c.acc = op;
        else if ((c.ir >> 4) == 0x1)
            c.acc = c.r[op];
        else
            c.acc = alu_out;
    }

    if (ctrl.REG_write)
        c.r[op] = c.acc;
}

```

```

}

/* ----- Loader ----- */

void load_program(string f)
{
    ifstream in(f);
    int x, i = 0;
    while (in >> hex >> x)
        mem[i++] = x;
}

/* ----- MAIN ----- */

int main()
{
    cpu c;
    open_log();

    while (true)
    {
        string cmd;
        cout << "Enter program file (RESET/EXIT): ";
        cin >> cmd;

        if (cmd == "RESET")
        {
            reset_cpu(c);
            continue;
        }
        if (cmd == "EXIT")
            return 0;

        load_program(cmd);
        break;
    }

    State state = FETCH;
    Control ctrl;

    while (state != HALT)
    {
        ctrl = Control();
        generate_control(c, state, ctrl);
        log_state(c, ctrl, state);
        apply_control(c, ctrl);

        if (state == FETCH)
            state = DECODE;
        else if (state == DECODE)
            state = EXEC;
        else if (state == EXEC)
            state = WB;
        else if (state == WB)

```

```

    {
        if (((c.ir >> 4) & 0xF) == 0xF)
            state = HALT;
        else
            state = FETCH;
    }
}
}

```

alu.hpp

```

#ifndef ALU_H
#define ALU_H

#include <stdint>

struct ALUResult
{
    uint8_t value; // result of operation
    bool Z; // zero flag
    bool C; // carry flag
};

/* ===== LOGIC GATES ===== */

inline bool AND(bool a, bool b) { return a & b; }
inline bool OR(bool a, bool b) { return a | b; }
inline bool XOR(bool a, bool b) { return a ^ b; }
inline bool NOT(bool a) { return !a; }

/* ===== ADDERS ===== */

struct HalfAdder
{
    bool sum;
    bool carry;
};

inline HalfAdder half_adder(bool a, bool b)
{
    return {XOR(a, b), AND(a, b)};
}

struct FullAdder
{
    bool sum;
    bool carry;
};

inline FullAdder full_adder(bool a, bool b, bool cin)

```

```

{
    HalfAdder h1 = half_adder(a, b);
    HalfAdder h2 = half_adder(h1.sum, cin);
    return {h2.sum, OR(h1.carry, h2.carry)};
}

/* ===== 8-BIT ADDER ===== */

inline uint8_t add8(uint8_t a, uint8_t b, bool &carry_out)
{
    uint8_t r = 0;
    bool carry = 0;

    for (int i = 0; i < 8; i++)
    {
        bool ai = (a >> i) & 1;
        bool bi = (b >> i) & 1;

        FullAdder f = full_adder(ai, bi, carry);

        if (f.sum)
            r |= (1 << i);
        carry = f.carry;
    }

    carry_out = carry;
    return r;
}

/* ===== TWO'S COMPLEMENT ===== */

inline uint8_t twos(uint8_t x)
{
    bool c;
    return add8(~x, 1, c);
}

/* ===== SUBTRACTION ===== */

inline uint8_t sub8(uint8_t a, uint8_t b, bool &borrow)
{
    bool c;
    uint8_t r = add8(a, twos(b), c);
    borrow = NOT(c);
    return r;
}

/* ===== MULTIPLY ===== */

inline uint8_t mul8(uint8_t a, uint8_t b, bool &carry)
{
    uint8_t r = 0;
    carry = 0;

```

```

    for (int i = 0; i < 8; i++)
    {
        if ((b >> i) & 1)
        {
            bool c;
            uint8_t shifted = a << i;
            r = add8(r, shifted, c);
            if (c)
                carry = 1;
        }
    }
    return r;
}

/* ===== DIVISION ===== */

inline uint8_t div8(uint8_t a, uint8_t b, bool &div0)
{
    if (b == 0)
    {
        div0 = true;
        return 0;
    }

    div0 = false;
    uint8_t q = 0;
    uint8_t r = a;

    for (int i = 7; i ≥ 0; i--)
    {
        uint8_t d = b << i;
        bool borrow;
        uint8_t tmp = sub8(r, d, borrow);

        if (!borrow)
        {
            r = tmp;
            q |= (1 << i);
        }
    }

    return q;
}

/* ===== POWER ===== */

inline uint8_t pow8(uint8_t a, uint8_t b, bool &carry)
{
    uint8_t r = 1;
    carry = 0;

    for (int i = 0; i < b; i++)
        r = mul8(r, a, carry);
}

```

```

        return r;
    }

/* ===== CPU-VISIBLE ALU ===== */

inline ALUResult alu_add(uint8_t a, uint8_t b)
{
    bool c;
    uint8_t r = add8(a, b, c);
    return {r, r == 0, c};
}

inline ALUResult alu_sub(uint8_t a, uint8_t b)
{
    bool br;
    uint8_t r = sub8(a, b, br);
    return {r, r == 0, br};
}

inline ALUResult alu_mul(uint8_t a, uint8_t b)
{
    bool c;
    uint8_t r = mul8(a, b, c);
    return {r, r == 0, c};
}

inline ALUResult alu_div(uint8_t a, uint8_t b, bool &div0)
{
    uint8_t r = div8(a, b, div0);
    return {r, r == 0, 0};
}

inline ALUResult alu_pow(uint8_t a, uint8_t b)
{
    bool c;
    uint8_t r = pow8(a, b, c);
    return {r, r == 0, c};
}

inline ALUResult alu_cmp(uint8_t a, uint8_t b)
{
    bool br;
    sub8(a, b, br);
    return {0, a == b, br};
}

#endif

```